第2章 缓存优化

2.1 存储体系基础知识复习

2.1.1 存储层次整体特性

计算机存储系统采用分层架构，各级存储器的速度、容量、价格呈梯度分布，核心适配程序局部性原理，平衡速度与成本。

速度排序：Cache（缓存）> 主存 > 辅存

容量排序：Cache < 主存 < 辅存

价格排序：Cache > 主存 > 辅存

典型访问时延：Cache（ns级）、主存（20ns左右）、辅存（ms级）

2.1.2 局部性原理（缓存核心理论）

程序执行时，指令和数据的访问具有局部性，是缓存能够高效工作的根本原因，分为两类：

时间局部性：刚被访问过的数据/指令，在短期内大概率会被再次访问（如循环代码、重复调用的变量）。

空间局部性：刚被访问的存储单元，其相邻的存储单元短期内大概率被访问（如数组连续遍历、顺序执行的指令）。

2.1.3 各类存储器硬件特性

SRAM（静态随机存储器）：用于Cache，由触发器构成，速度极快、无需刷新、成本高、集成度低。

DRAM（动态随机存储器）：用于主存，由电容储能实现，速度中等、需要定期刷新、集成度高、成本适中。

闪存/EEPROM/磁盘：用于辅存，容量极大、速度慢、成本极低，用于长期数据存储。

2.1.4 Cache核心工作机制

1. 主存–Cache映射方式

直接映射：一个主存块只能固定映射到Cache**唯一指定位置**，硬件简单、速度快，冲突不命中多。

全相联映射：主存块可映射到Cache**任意空闲位置**，无冲突不命中，硬件复杂、查找延迟高、成本高。

组相联映射：折中方案，将Cache分为若干组，主存块固定映射到指定组，组内全相联，兼顾速度与命中率（最常用）。

1. 直白定义

- **相联度 n**：**同一个组里，最多能放 n 个 Cache 块**
- 又叫：**n 路组相联**

举例：

- 1 路 → 直接映射（一组只能放 1 块）
- 2 路组相联 → **一组里面能放 2 块**
- 4 路组相联 → **一组里面能放 4 块**

2. 分清两个东西（必考）

1. **组数**：Cache 一共分成多少个大组
2. **相联度（路数）**：**每一组内部有多少块**

公式

$$\text{总块数} = \text{组数} \times \text{相联度}$$

不管是直接映像、组相联还是全相联，地址都要拆成三段（按从高到低的顺序）：

`[Tag（标识）] + [Index（索引）] + [Block Offset（块内偏移）]`

它们的作用是：

- **Block Offset（块内偏移）**：用来定位 Cache 块里的具体字节
- **Index（索引）**：用来定位 Cache 里的哪一组（组相联）或哪一块（直接映像）
- **Tag（标识）**：用来判断 Cache 里的块是不是你要找的那个

2. 替换算法

Cache块满、需要调入新块时，替换旧块的规则：

随机替换：硬件简单，无优化逻辑，性能一般。

LRU（最近最少使用）：替换最久未被访问的块，贴合局部性原理，命中率最高，工业界主流。

3. Cache写策略

针对CPU写数据场景，分写命中、写不命中两类场景：

写命中（数据块已在Cache中）

写直达：同时写入Cache和主存，数据一致性高，速度慢。

写回：仅写入Cache，替换时才写回主存，速度快，存在数据不一致风险。

写不命中（数据块不在Cache中）

写分配：先将主存块调入Cache，再写入数据，利用空间局部性。

写不分配：直接写入主存，不调入Cache，避免无效块占用缓存。

2.1.5 虚拟存储与TLB机制

地址转换流程：CPU虚拟地址（逻辑地址）→ TLB（快表）→ 物理地址；TLB缺失则查询页表。

核心状态：TLB命中（快速完成地址转换）、TLB缺失、页面缺页（需从辅存调页）。

页式管理：虚拟内存分页存储，解决内存空间不足、地址不连续问题。

> **第 1 步：CPU 给出 虚拟地址**
>
> 你现在只有虚拟地址，**内存不认，Cache 也不认。** 必须先转成 **物理地址**。
>
> **第 2 步：先查 快表 TLB（地址转换专用缓存）**
>
> TLB 里面存的是：**虚拟页号 → 物理页号**
>
> **情况 A：TLB 命中 ✅**
>
> → **直接拿到物理地址** → 地址转换完成！
>
> **情况 B：TLB 缺失 ❌**
>
> → 去**主存查页表** → 查到后，得到物理地址 → 同时把这条映射塞回 TLB（方便下次快）
>
> **第 3 步：查完页表，才会判断 缺不缺页**
>
> 页表里有一个标记位：**有效位 / 在内存位**
>
> **情况 1：页在内存里 ✅**
>
> → 正常，继续访问

> **情况 2：页 不在内存 ❌**
>
> → **触发缺页中断（Page Fault）** → 去硬盘把这一页读进内存 → 更新页表 → 再重新访问
>
> **第 4 步：现在有了 物理地址**
>
> 才开始访问 Cache！
>
> **Cache 命中 ✅**
>
> → 直接读数据，结束
>
> **Cache 缺失 ❌**
>
> → 去**主存**读数据 → 把数据装入 Cache → 返回数据

2.2 Cache性能分析与改进（核心计算考点）

2.2.1 核心性能公式

1. 平均访存时间

平均访存时间 = 命中时间 + 不命中率 × 不命中开销

命中时间：Cache命中时的访问延迟（时钟周期/时间单位）。

不命中率（失效率）：访存不命中次数/总访存次数。

不命中开销：不命中时，从下级存储器调块的延迟。

2. 程序CPU执行时间

CPU时间 =（CPU执行周期数 + 存储器停顿周期数）× 时钟周期时间

推导公式：**存储器停顿时钟周期数 = 访存次数 × 不命中率 × 不命中开销**

通用简化公式：**CPU时间 = IC × (CPI执行 + 单指令平均访存次数 × 不命中率 × 不命中开销) × 时钟周期**

IC = Instruction Count → 指令条数

CPI 执行 = Cycles Per Instruction → 每条指令执行周期数

2.2.2 经典例题解析（高频考题）

例题1：基础Cache性能计算

已知：Cache不命中开销50个时钟周期，无缓存停顿CPI=2.0，不命中率2%，单指令平均访存1.33次。求实际CPI与性能变化。

解：实际CPI = 2.0 + 1.33 × 2% × 50 = 3.33

性能变化：3.33/2.0 = 1.67倍（CPU时间增加为原来1.67倍）

无Cache场景CPI：2.0 + 50×1.33 = 68.5，体现Cache对性能的巨大提升。

例题2：直接映像VS两路组相联性能对比

已知条件：

理想Cache（100%命中）CPI=2.0，基础时钟周期2ns，单指令访存1.3次；

两种Cache均为64KB、块大小32B；

两路组相联时钟周期放大1.1倍（硬件延迟）；

不命中开销70ns，命中时间1个时钟周期，即第一个条件的2ns；

直接映像不命中率1.4%，两路组相联不命中率1.0%。

计算平均访存时间：

直接映像：2.0 + 0.014×70 = 2.98ns

两路组相联：2.0×1.1 + 0.010×70 = 2.90ns

$$\text{CPU时间}_{1路} = IC\times(2.0\times2+(1.3\times0.014\times70) = 5.27\times IC$$
$$\text{CPU时间}_{2路} = IC\times(2.0\times2\times1.10+(1.3\times0.010\times70)) = 5.31\times IC$$

$$\frac{\text{CPU时间}_{2路}}{\text{CPU时间}_{1路}} = \frac{5.31\times IC}{5.27\times IC} = 1.01$$

2.2.3 关键性能结论

Cache对**低CPI、高时钟频率**的CPU至关重要

CPIexecution越低，固定周期数的Cache不命中开销的相对影响就越大。

在计算CPI时，不命中开销的单位是时钟周期数。因此，即使两台计算机的存储层次完全相同，时钟频率较高的CPU的不命中开销较大，其CPI中存储器停顿这部分也就较大。

Cache性能三大核心影响指标：命中时间、不命中率、不命中开销。

2.3 Cache不命中分类（3C分类法）

2.3.1 三类不命中定义与特性

1. 强制性不命中（Compulsory Miss）

别称：冷启动不命中、**首次访问不命中**

成因：首次访问某主存块，该块从未调入Cache，必须从下级存储器加载。

核心特性：**不受Cache容量、相联度影响**，所有Cache都无法完全避免。

2. 容量不命中（Capacity Miss）

成因：程序运行所需数据块总量超过Cache最大容量，**旧块被替换后再次被访问，引发不命中。**

核心特性：**不受相联度影响**；Cache容量越大，容量不命中越少。

3. 冲突不命中（Conflict Miss）

别称：碰撞不命中、干扰不命中

成因：直接/组相联Cache中，多个主存块映射到同一Cache组/块，导致有效块被替换，后续重新访问时失效（其他组仍有空闲空间）。

核心特性：**相联度越高，冲突不命中越少**；全相联无冲突不命中。

2.3.2 三类不命中影响因素总结表

不命中类型	容量影响	相联度影响

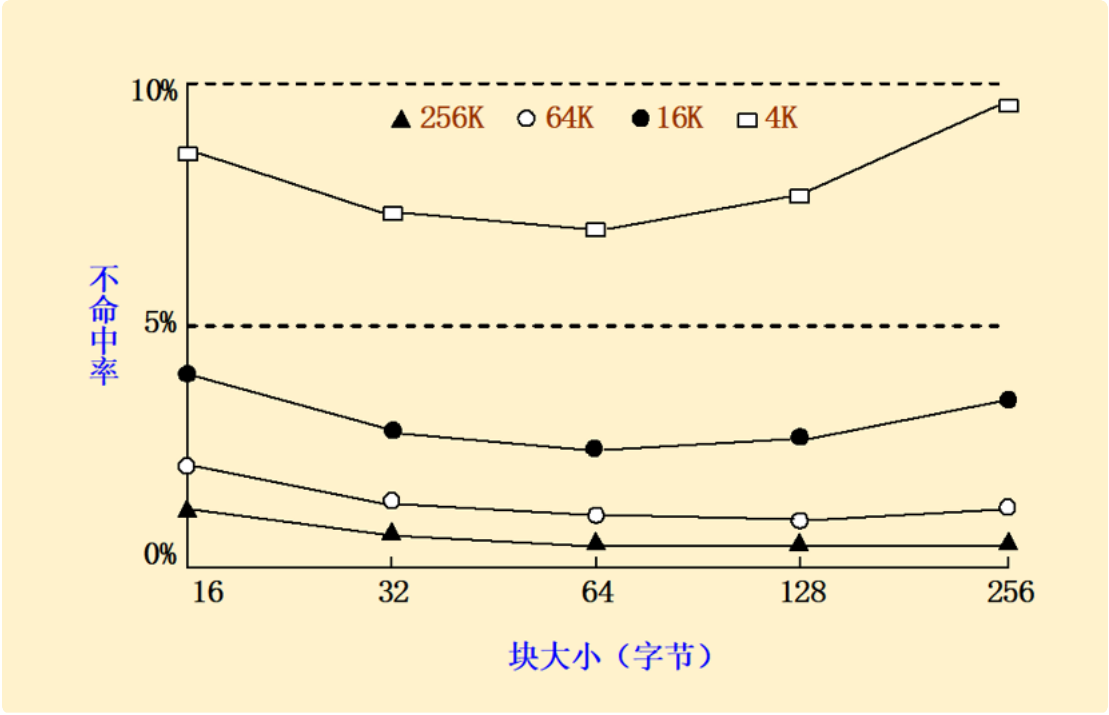
强制性不命中	无影响	无影响
容量不命中	容量增大，不命中减少	无影响
冲突不命中	容量增大，不命中略有减少	相联度越高，不命中越少

2.4 降低Cache不命中率（8种方法）

方法一：针对强制不命中 —— 增加Cache块大小

优化原理：利用空间局部性，单次调入更多相邻数据，大幅减少强制性不命中。

性能规律：固定Cache容量下，不命中率随块大小增大**先降后升**。



利弊分析

优点：有效降低强制性不命中；Cache容量越大，最优块大小越大。

缺点：**块过大导致Cache总块数减少，冲突不命中上升；不命中开销显著增加（调块数据量变大）。**

方法二：针对容量不命中 —— 增加Cache容量

优化原理：容纳更多程序数据块，彻底减少容量不命中。

利弊分析

缺点：硬件成本升高、**Cache查找路径变长，命中时间增加**；多用于片外二级Cache。

方法三：针对冲突不命中 -- 提高相联度

优化原理：缓解多块映射冲突，大幅降低冲突不命中。

核心经验规则：N容量直接映像Cache ≈ N/2容量两路组相联Cache 的不命中率。

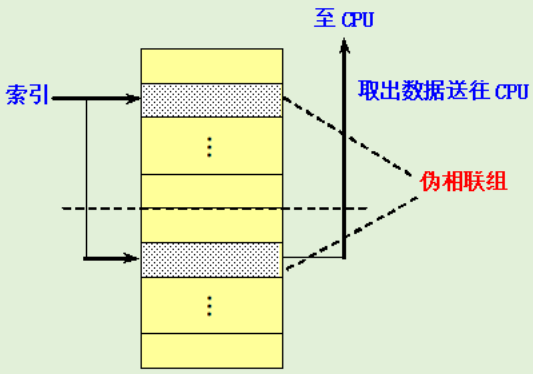
利弊：相联度越高，命中时间增加；相联度超过8后，优化收益极低，无实际意义。

方法四：伪相联Cache（折中最优）

- 多路组相联 V.S. 直接映像

	优 点	缺 点
直接映像	命中时间小	不命中率高
组相联	不命中率低	命中时间大

- 伪相联Cache的优点
  - 命中时间小
  - 不命中率低



核心思想：结合直接映像（快）和组相联（低失效率）的优点，将直接映像Cache空间平分为两个区域。

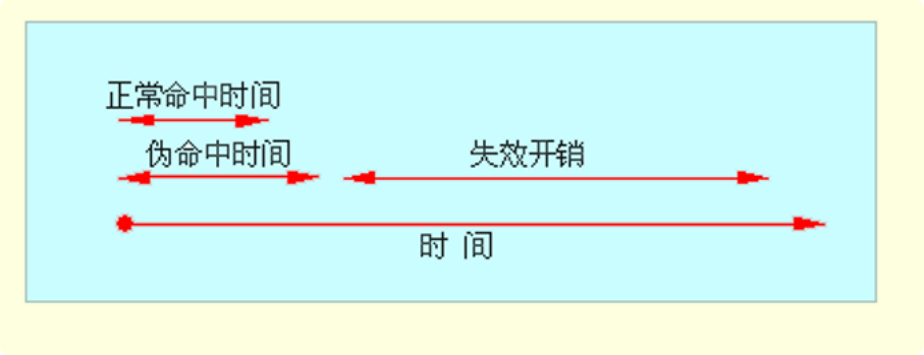
工作流程

优先按直接映像方式访问，命中则快速返回；

不命中则查找另一分区对应位置，命中为**伪命中**（轻微延迟）；

仍不命中，再访问下级存储器。

优缺点：命中时间接近直接映像，不命中率接近两路组相联；**存在快速命中、慢速伪命中两种时延，需保证绝大多数为快速命中。**



结论：中小容量Cache场景下，伪相联Cache平均访存时间优于直接映像和两路组相联。

例题

解　平均访存时间$_{伪相联}$ = ① 命中时间$_{伪相联}$ + ② 失效率$_{伪相联}$ × ③ 失效开销$_{伪相联}$

① 命中时间$_{伪相联}$

命中时间$_{伪相联}$ = 命中时间$_{1路}$ + 伪命中率$_{伪相联}$ × 2

伪相联查找的命中率等于2路组相联Cache的命中率和直接映象Cache命中率之差。

伪命中率$_{伪相联}$ = 命中率$_{2路}$ − 命中率$_{1路}$
= （1 − 失效率$_{2路}$） − （1 − 失效率$_{1路}$）
= 失效率$_{1路}$ − 失效率$_{2路}$

② 失效率$_{伪相联}$ = 失效率$_{2路}$

③ 失效开销$_{伪相联}$ = 失效开销$_{1路}$

综合上述分析，有：

平均访存时间$_{伪相联}$ = 命中时间$_{1路}$ + （失效率$_{1路}$ − 失效率$_{2路}$） × 2 + 失效率$_{2路}$ × 失效开销$_{1路}$

例　假设命中时间为1个周期，按直接映象找到的位置处没有发现匹配，而在另一个位置才找到数据（伪命中）需要2个额外的周期。问：当Cache容量分别为2 KB和128 KB时，直接映象、2路组相联和伪相联这三种组织结构中，哪一种速度最快？

Cache容量为2KB时，有：

	失效率	失效开销（周期）	平均访存时间（周期）
1路	0.098	50	5.90
2路	0.076	50	4.90

Cache容量为128KB时，有：

	失效率	失效开销（周期）	平均访存时间（周期）
1路	0.010	50	1.50
2路	0.007	50	1.45

将前面表中的数据代入上面的公式，得：

$$\text{平均访存时间}_{\text{伪相联，2 KB}} = 1 + (0.098 - 0.076) \times 2 + (0.076 \times 50) = 4.844$$

对于2 KB Cache，有：

$$\text{平均访存时间}_{\text{1路}} = 5.90 \text{ 个时钟}$$

$$\text{平均访存时间}_{\text{2路}} = 4.90 \text{ 个时钟}$$

对于128KB的Cache有，可得：

$$\text{平均访存时间}_{\text{伪相联，128 KB}} = 1 + (0.010 - 0.007) \times 2 + (0.007 \times 50) = 1.356$$

$$\text{平均访存时间}_{\text{1路}} = 1.50 \text{ 个时钟}$$

$$\text{平均访存时间}_{\text{2路}} = 1.45 \text{ 个时钟}$$

可见，对于这两种Cache容量，伪相联Cache都是速度最快的。

方法五：硬件预取

原理：硬件自动**提前预取指令/数据到Cache或流缓冲器**，利用空间局部性消除后续不命中。

分类：指令预取、数据预取，预取内容可放入Cache或独立流缓冲器。

核心要求：预取必须利用存储器空闲带宽，不能影响正常访存，否则会降低性能。

方法六：编译器控制预取

原理：**编译阶段**手动**插入预取指令**，**在数据使用前提前加载**，让访存与CPU执行重叠。

适用场景：循环遍历数组（高频不命中场景），需提前若干循环周期预取。

分类：

按照预取数据所放的位置：

寄存器预取：把数据取到寄存器中。

Cache预取：只将数据取到Cache中。

按照预取的处理方式不同：

故障性预取：在预取时，若出现虚地址故障或违反保护权限，就会发生异常。

非故障性预取：在遇到这种情况时则不会发生异常，因为这时它会放弃预取，转变为空操作。

本节假定Cache预取都是非故障性的，也叫做非绑定预取。

> - 在预取数据的同时，处理器应能继续执行。只有这样，预取才有意义。
>
> 非阻塞 Cache（非锁定 Cache）： Cache 不命中时，CPU 可以继续执行后面不依赖这块数据的指令，不用等。
>
> - 编译器控制预取（就是在编译时插入预取指令），目标就是：让 CPU 执行指令的时间，和数据从内存里读回来的时间，重叠起来。
>
> - 循环是预取优化的主要对象
>
> 不命中开销小时：循环体展开 1～2 次；不命中开销大时：循环体展开许多次
>
> 核心逻辑：不命中越贵，越要提前预取，越需要通过循环展开来安排预取时机。
>
> - 优化原则：预取指令开销必须小于不命中开销，避免得不偿失。
>
> - 编译器可以通过把重点放在那些可能会导致不命中的访问上，使程序避免不必要的预取，从而较大程度地减少平均访存时间

方法七：编译器软件优化（无硬件改动）

- 基本思想：通过对软件进行优化来降低不命中率。

  （特色：无需对硬件做任何改动）

- 程序代码和数据重组
  - 可以重新组织程序而不影响程序的正确性
    - 把一个程序中的过程重新排序，就可能会减少冲突不命中，从而降低指令不命中率。
      - McFarling研究了如何使用配置文件（profile）来进行这种优化。
    - 把基本块对齐，使得程序的入口点与Cache块的起始位置对齐，就可以减少顺序代码执行时所发生的Cache不命中的可能性。
  - 如果编译器知道一个分支指令很可能会成功转移，那么它就可以通过以下两步来改善**空间局部性**：
    - 将转移目标处的基本块和紧跟着该分支指令后的基本块进行对调；
    - 把该分支指令换为操作语义相反的分支指令。
  - 数据对存储位置的限制更少，更便于调整顺序。

这两点都是使得大概率执行的代码段放在分支失败处

纯软件优化，无需修改硬件，通过调整代码结构提升局部性，减少不命中：

数组合并：将独立且同步访问的数组合并为结构体数组，提升空间局部性。

举例：

```
/* 修改前 */
int x[100];
int y[100];

  若x[i]和y[i]经常一起访问
```

```
/* 修改后 */
struct merge{
    int x;
    int y;
};
struct merge merged_array[100];
```

内外循环交换：调整循环顺序，将列访问改为行访问，适配数组连续存储特性。

举例：

```
/* 修改前 */
for （j = 0；j < 100；j = j+1 ）
  for （i = 0；i < 5000；i = i+1 ）
    x[i][j] = 2 * x[i][j];
```

```
/* 修改后 */
for （i = 0；i < 5000；i = i+1 ）
  for （j = 0；j < 100；j = j+1 ）
    x[i][j] = 2 * x[i][j];
```

循环融合：将多个访问相同数据的独立循环合并，复用Cache中的数据块。

举例：

```
/* 修改前 */
for (j = 0; j < 100; j = j+1 )
   x[i][j] = a[i][j] + b[i][j];
for (j = 0; j < 100; j = j+1 )
   y[i][j] = a[i][j] - b[i][j];
```

```
/* 修改后 */
for (j = 0; j < 100; j = j+1 ) {
   x[i][j] = a[i][j] + b[i][j];
   y[i][j] = a[i][j] - b[i][j];
}
```

分块优化：将大规模数组整行/整列访问改为分块访问，减少Cache块频繁替换。

```
/* 修改前 */
for （i = 0; i<N; i = i+1 ）
   for （j = 0; j < N; j = j+1 ） {
      r = 0;
      for （k = 0; k < N; k = k+1） {
         r = r + y[i][k] * z[k][j];
   }
      x[i][j] = r;
}
```

```
/* 修改后 */
for （jj = 0; jj < N; jj = jj+B ）
for （kk = 0; kk < N; kk = kk+B ）
for （i = 0; i < N; i =i+1 ）
for （j = jj; j < min（jj+B-1, N）; j = j+1 ） {
  r = 0;
  for （k = kk; k < min（kk+B-1, N）; k = k+1）
    {
   r = r + y[i][k] * z[k][j];
  }
   x[i][j] = x[i][j] + r;
 }
```

方法八：牺牲Cache（Victim Cache）

原理：在主Cache和下级存储器之间，增设一个**小型全相联Cache**，专门存放被主Cache替换出去的“牺牲块”。

作用：针对**冲突不命中**优化效果极强，尤其适配小容量直接映像Cache，可减少20%~90%的冲突不命中。

优势：不影响CPU时钟频率，硬件开销小。

2.5 降低Cache不命中开销（5种方法）

核心目标：缩短Cache不命中后，访问下级存储器的延迟。

方法一：两级Cache（多级Cache）

1. 架构设计

L1（一级Cache）：小而快，集成在CPU片内，匹配主频；

L2（二级Cache）：大容量、速度适中，承接L1不命中请求。

2. 性能计算公式

$$\text{平均访存时间} = \text{命中时间}_{L1} + \text{不命中率}_{L1} \times \text{不命中开销}_{L1}$$

$$\text{不命中开销}_{L1} = \text{命中时间}_{L2} + \text{不命中率}_{L2} \times \text{不命中开销}_{L2}$$

$$\text{平均访存时间} = \text{命中时间}_{L1} + \text{不命中率}_{L1} \times (\text{命中时间}_{L2} + \text{不命中率}_{L2} \times \text{不命中开销}_{L2})$$

不命中率L2 -> 局部不命中率

3. 局部不命中率VS全局不命中率

局部不命中率：本级Cache不命中次数 / 到达本级的访存次数；

全局不命中率：本级Cache不命中次数 / CPU总访存次数（**性能评价核心指标**）；

全局不命中率L2 = L1不命中率 × L2局部不命中率。

- 采用两级Cache时，每条指令的平均访存停顿时间：

$$\text{每条指令的平均访存停顿时间} = \text{每条指令的平均不命中次数}_{L1} \times \text{命中时间}_{L2} + \text{每条指令的平均不命中次数}_{L2} \times \text{不命中开销}_{L2}$$

假设：

- 每条指令平均访存次数：1 次
- L1 不命中率：5%
- L2 局部不命中率：10%
- L2 命中时间：10 个时钟周期
- L2 不命中开销（去主存）：100 个时钟周期

套公式：

1. 第一部分：L1 不命中带来的停顿
   `1 × 5% × 10 = 0.5 个周期`
2. 第二部分：L2 不命中带来的停顿
   `(1 × 5% × 10%) × 100 = 0.5 个周期`
3. 总停顿时间：`0.5 + 0.5 = 1 个周期/指令`

例题

例

考虑某一两级Cache：第一级Cache为L1，第二级Cache为L2。

（1）假设在1000次访存中，L1的不命中是40次，L2的不命中是20次。求各种局部不命中率和全局不命中率。

（2）假设L2的命中时间是10个时钟周期，L2的不命中开销是100时钟周期，L1的命中时间是1个时钟周期，平均每条指令访存1.5次，不考虑写操作的影响。问：平均访存时间是多少？每条指令的平均停顿时间是多少个时钟周期？

解 （1）第一级Cache的不命中率（全局和局部）是40/1000，即4%；

第二级Cache的局部不命中率是20/40，即50%；

第二级Cache的全局不命中率是20/1000，即2%。

（2）平均访存时间＝命中时间$_{L1}$＋不命中率$_{L1}$×（命中时间$_{L2}$＋不命中率$_{L2}$×不命中开销$_{L2}$）

＝1＋4%×（10＋50%×100）

＝1＋4%×60＝3.4个时钟周期

由于平均每条指令访存1.5次，且每次访存的平均停顿时间为：

3.4－1.0＝2.4

所以：

每条指令的平均停顿时间＝2.4×1.5＝3.6个时钟周期

- 对于第二级Cache，我们有以下结论：
  - 在第二级Cache比第一级 Cache大得多的情况下，两级Cache的全局不命中率和容量与第二级Cache相同的单级Cache的不命中率非常接近。
  - 局部不命中率不是衡量第二级Cache的一个好指标，因此，在评价第二级Cache时，应用全局不命中率这个指标。
- 第二级Cache不会影响CPU的时钟频率，因此其设计有更大的考虑空间。
  - 设计第二级Cache时有两个问题需要权衡：
    - 它能否降低CPI中的平均访存时间部分？
    - 它的成本是多少？
- 第二级Cache的参数
  - 容量：第二级Cache的容量一般比第一级的大许多。
    - 大容量意味着第二级Cache可能实际上没有容量不命中，只剩下一些强制性不命中和冲突不命中。
  - 相联度：第二级Cache可采用较高的相联度或伪相联方法。

- 块大小
  - 第二级Cache可采用较大的块，如 64、128、256字节
  - 为减少平均访存时间，可以让容量较小的第一级Cache采用较小的块，而让容量较大的第二级Cache采用较大的块。
  - 多级包容性

    需要考虑的另一个问题：

    第一级Cache中的数据是否总是同时存在于第二级Cache中。如果是，就说第二级Cache是具有多级包容性的。

方法二：读不命中优先于写

问题场景：**写缓冲器中存在未写入主存的最新数据，读不命中时可能读取到旧主存数据。**

解决方案：推迟对读不命中的处理，直到写缓冲器清空（缺点：读不命中的开销增加）

检查写缓冲器中的内容，若无相同，且存储器可用，继续处理读不命中（常用方案）

方法三：写缓冲合并

原理：合并写缓冲器中同地址、相邻地址的写操作，将多次小写入合并为一次大块写入。

作用：提升写缓冲空间利用率，减少写缓冲满阻塞，大幅提升连续写操作效率。

- 写直达Cache依靠写缓冲来减少对下一级存储器写操作的时间。
  - 如果写缓冲器为空，就把数据和相应地址写入该缓冲器。从CPU的角度来看，该写操作就算是完成了。
  - 如果写缓冲器中已经有了待写入的数据，就要把这次的写入地址与写缓冲器中已有的所有地址进行比较，看是否有匹配的项。如果有地址匹配而对应的位置又是空闲的，就把这次要写入的数据与该项合并。这就叫写缓冲合并。
  - 如果写缓冲器满且又没有能进行写合并的项，就必须等待。
- 作用
  - 连续写入多个字的速度快于每次只写入一个字的操作；
  - 提高了写缓冲器的空间利用率，减少因写缓冲器满而要进行的等待时间。

例如　写缓冲区有4项，每项4个64位的字。

写地址	V		V		V		V	
100	1	Mem[100]	0		0		0	
108	1	Mem[108]	0		0		0	
116	1	Mem[116]	0		0		0	
124	1	Mem[124]	0		0		0	

（a）不采用写合并

写地址	V		V		V		V	
100	1	Mem[100]	1	Mem[108]	1	Mem[116]	1	Mem[124]
	0		0		0		0	
	0		0		0		0	
	0		0		0		0	

（b）采用了写合并

方法四：请求字处理技术

请求字：Cache不命中时，CPU当前急需的单个数据字。

优化策略

尽早重启动：**请求字到达后，立即发送给CPU**，CPU无需等待整块数据加载完成；

请求字优先：**存储器优先返回请求字**，后台继续加载剩余块数据

在以下情况下，用不用差别不大：Cache块较小/下一条指令正好访问同一Cache块的另一部分

方法五：非阻塞Cache（乱序Cache）

原理：**Cache不命中时，不阻塞CPU，允许CPU乱序执行其他指令，支持同时处理多次不命中。**

性能特点：

- 不是**可以同时处理不命中的次数**越多越好，要看程序类型：
- 浮点程序：受益明显，越多越好。
- 整数程序：提升很快就饱和了，1–2 次就几乎拿到了所有好处。
- 对整数程序来说，“一次不命中下命中”（也就是处理 1 个重叠不命中）就足够了，没必要做特别复杂的多不命中支持。

2.6 减少Cache命中时间（4种方法）

命中时间直接限制CPU的时钟频率，是高频CPU的核心优化点。

方法一：小容量、简单结构Cache

硬件越简单、容量越小，查找延迟越低；优先采用直接映像Cache，适配CPU片内集成，保证极致速度。

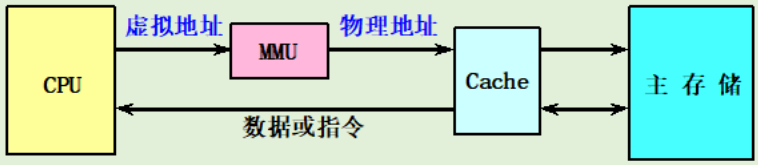
某些设计采用了一种折衷方案：把Cache的标识放在片内，而把Cache的数据存储器放在片外。

方法二：虚拟Cache与虚实混合Cache

1. 物理Cache

基于物理地址访问，需先完成虚实地址转换再访问Cache，串行操作，速度慢。

- 物理Cache
  - 使用物理地址进行访问的传统Cache。
  - 标识存储器中存放的是物理地址，进行地址检测也是用物理地址。
  - 缺点：地址转换和访问Cache串行进行，访问速度很慢。

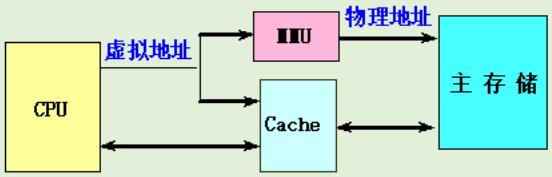


物理Cache存储系统

2. 虚拟Cache

直接用虚拟地址访问Cache，命中时无需地址转换，速度极快；即使不命中，地址转换和访问Cache也是并行进行的，其速度比物理Cache快很多。

- 虚拟Cache
  - 可以直接用虚拟地址进行访问的Cache。标识存储器中存放的是虚拟地址，进行地址检测用的也是虚拟地址。
  - 优点：在命中时不需要地址转换，省去了地址转换的时间。即使不命中，地址转换和访问Cache也是并行进行的，其速度比物理Cache快很多。



问题：

- 并非都采用虚拟Cache
  - 清空问题：由于新进程的虚拟地址可能与原进程相同；
  - 解决方法：在地址标识中增加PID(进程标识符)字段；
  - PID相关问题：为了减少位数，PID可能循环使用，所以也可能存在多个进程使用同一个PID，所以此时也需要清空Cache。

3. 虚拟索引+物理标识

一、先把地址拆明白：虚拟地址和物理地址的结构

不管是虚拟地址还是物理地址，都会被分成两部分：

- **高位：页号（Virtual/Physical Page Number）**
- **低位：页内位移（Offset）**

举个例子：假设页大小是 4KB

- 页大小 4KB = 2^{12} 字节 → 页内位移占**低 12 位**
- 剩下的高位，就是页号（虚拟页号 VPN、物理页号 PPN）

所以，地址的结构是这样的：

表格

地址类型	高位（页号）	低位（页内位移，12 位）
虚拟地址	虚拟页号 VPN	页内位移 Offset
物理地址	物理页号 PPN	页内位移 Offset

二、关键结论：转换只改「页号」，不改「页内位移」

页表（Page Table）里存的是：
`虚拟页号VPN → 物理页号PPN` 的映射关系。

地址转换的过程是：

1. 拿到虚拟地址，把它拆成 `VPN + Offset`
2. 用 VPN 查页表，得到对应的 PPN
3. 把 PPN 和原来的 Offset 拼起来，就是物理地址：`PPN + Offset`

所以，转换前后，页内位移 Offset 是完全不变的！

用虚拟地址页内偏移做Cache索引，用物理地址做标识校验，兼顾虚拟Cache的速度和物理Cache的稳定性。

限制：Cache容量 ≤ 页大小 × 相联度。

- 页大小＝4KB　　相联度＝16



Cache容量＝16×4KB＝64KB

方法三：Cache访问流水化

将Cache访问过程拆分为多级流水线，缩短单周期访问延迟，提升主频（如Pentium4采用4级流水访问Cache）。

方法四：踪迹Cache（指令Cache，负责存指令而不是数据）

开发指令级并行性所遇到的一个挑战是：当要每个时钟周期流出超过4条指令时，要提供足够多条彼此互不相关的指令是很困难的。

PPT 里说：

> 存放 CPU 所执行的动态指令序列，包含了由分支预测展开的指令。

- 传统指令 Cache 存的是**静态的、按地址顺序排列的指令块。**
- 踪迹 Cache 存的是**CPU 实际执行过的动态指令序列（Trace）**，它是按“执行顺序”存的，而不是“地址顺序”。

举个例子：
假设程序执行流是：`指令A → 条件分支 → 分支预测成功走了“分支1”的指令B → 指令C`

- 传统 Cache 里，A、B、C 是按地址分散存的，分支跳转后要重新取 B 和 C，效率低。
- 踪迹 Cache 里，会把 `A → B → C` 这一段实际执行过的序列，打包成一个“踪迹”存在一起。下次再遇到同样的分支预测结果，CPU 可以一次性取出这一整条踪迹，喂饱多个发射端口。

1. **CPU 执行指令时，把实际执行的序列打包成“踪迹”**：
   比如，把一条分支预测成功的路径上的所有指令，打包成一个连续的踪迹块。
2. **把踪迹存在踪迹 Cache 里**：下次再遇到相同的 PC 和分支预测结果，直接从 Cache 里取整条踪迹，不用再解码和处理分支。
3. **分支预测需要验证**：PPT 里说“该分支预测是否正确需要在取到该指令时进行确认”。如果预测错了，就要丢弃这个踪迹，重新取指令。

优点
提高指令 Cache 的空间利用率：把分散的指令按执行顺序打包，减少了分支跳转带来的 Cache 碎片问题，一次能取够多条指令。
能一次性给宽发射 CPU 提供足够多的连续指令，大幅提升指令级并行效率。
缺点
地址映像机制复杂：踪迹不是按静态地址存的，索引和查找逻辑比传统 Cache 复杂得多。
相同指令序列可能被重复存放：比如同一个分支的“走 / 不走”两种路径，会被当成两个不同的踪迹存起来，导致同一段指令序列被重复存储，浪费空间。

2.7 全书Cache优化技术总结

符号说明：+优化指标、–恶化指标、无影响为空；复杂度0（最简单）~3（最复杂）

Cache优化技术总结

优化技术	不命中率	不命中开销	命中时间	硬件复杂度	说 明
增加块大小	+	–		0	实现容易；Pentium 4 的第二级Cache采用了128字节的块
增加Cache容量	+			1	被广泛采用，特别是第二级Cache
提高相联度	+		–	1	被广泛采用
牺牲Cache	+			2	AMD Athlon采用了8个项的Victim Cache
伪相联Cache	+			2	MIPS R10000的第二级Cache采用
硬件预取指令和数据	+			2～3	许多机器预取指令，UltraSPARC III预取数据

优化技术	不命中率	不命中开销	命中时间	硬件复杂度	说 明
编译器控制的预取	+			3	需同时采用非阻塞Cache；有几种微处理器提供了对这种预取的支持
用编译技术减少Cache不命中次数	+			0	向软件提出了新要求；有些机器提供了编译器选项
使读不命中优先于写		+	–	1	在单处理机上实现容易，被广泛采用
写缓冲归并		+		1	与写直达合用，广泛应用，例如21164，UltraSPARC III
尽早重启动和关键字优先		+		2	被广泛采用
非阻塞Cache		+		3	在支持乱序执行的CPU中使用

优化技术	不命中率	不命中开销	命中时间	硬件复杂度	说 明
两级Cache			+	2	硬件代价大；两级Cache的块大小不同时实现困难；被广泛采用
容量小且结构简单的Cache	–		+	0	实现容易，被广泛采用
对Cache进行索引时不必进行地址变换			+	2	对于小容量Cache来说实现容易，已被Alpha 21164和UltraSPARC III采用
流水化Cache访问			+	1	被广泛采用
Trace Cache			+	3	Pentium 4 采用

两级Cache：标错了 应是不命中开销+